

Sol1:

```
#include <iostream>
using namespace std;

struct Node {
    char data;
    Node* prev;
    Node* next;
    Node(char val) {
        data = val;
        prev = next = NULL;
    }
};

class DoublyLinkedList {
    Node* head;
    Node* tail;
    int size;

public:
    DoublyLinkedList() {
        head = tail = NULL;
        size = 0;
    }

    void insertEnd(char val) {
        Node* newNode = new Node(val);
        if (!head) {
            head = tail = newNode;
        }
        else {
            tail->next = newNode;
            newNode->prev = tail;
            tail = newNode;
        }
        size++;
    }

    void insertBegin(char val) {
        Node* newNode = new Node(val);
        if (!head) {
            head = tail = newNode;
        }
        else {
            newNode->next = head;
        }
    }
};
```

```

        head->prev = newNode;
        head = newNode;
    }
    size++;
}

void insertBeforePos(int pos, char val) {
    if (pos < 1 || pos > size + 1) {
        cout << "Invalid position: " << pos << endl;
        return;
    }

    if (pos == 1) {
        insertBegin(val);
        return;
    }

    Node* temp = head;
    for (int i = 1; i < pos; i++) {
        temp = temp->next;
    }

    Node* newNode = new Node(val);
    newNode->next = temp;
    newNode->prev = temp->prev;
    temp->prev->next = newNode;
    temp->prev = newNode;
    size++;
}

void deleteBegin() {
    if (!head) return;
    Node* temp = head;
    head = head->next;
    if (head) head->prev = NULL;
    else tail = NULL;
    delete temp;
    size--;
}

void deleteEnd() {
    if (!tail) return;
    Node* temp = tail;
    tail = tail->prev;
    if (tail) tail->next = NULL;
    else head = NULL;
    delete temp;
    size--;
}

```

```

void deletePos(int pos) {
    if (pos < 1 || pos > size) {
        cout << "Invalid deletion position: " << pos << endl;
        return;
    }

    if (pos == 1) {
        deleteBegin();
        return;
    }

    if (pos == size) {
        deleteEnd();
        return;
    }

    Node* temp = head;
    for (int i = 1; i < pos; i++) {
        temp = temp->next;
    }

    temp->prev->next = temp->next;
    temp->next->prev = temp->prev;
    delete temp;
    size--;
}

void display() {
    Node* temp = head;
    cout << "List (" << size << " elements): ";
    while (temp) {
        cout << temp->data << " ";
        temp = temp->next;
    }
    cout << endl;
}

int getSize() { return size; }
};

int main() {
    DoublyLinkedList dll;

    cout << "Inserting A-J at end: ";
    for (char ch = 'A'; ch <= 'J'; ch++) {
        dll.insertEnd(ch);
        cout << ch << " ";
    }
    cout << endl;
    dll.display();

    cout << "Inserting K-T at beginning: ";
    for (char ch = 'T'; ch >= 'K'; ch--) {

```

```

        dll.insertBegin(ch);
        cout << ch << " ";
    }
    cout << endl;
    dll.display();

    int positions[] = { 4, 8, 12, 16, 20 };
    char chars[] = { 'U', 'V', 'W', 'X', 'Y' };

    cout << "Inserting U-Y before positions 4,8,12,16,20: ";
    for (int i = 0; i < 5; i++) {
        dll.insertBeforePos(positions[i], chars[i]);
        cout << chars[i] << " before " << positions[i] << " ";
    }
    cout << endl;

    dll.insertEnd('Z');
    cout << "Inserting Z at end" << endl;

    cout << "Final list after all insertions: ";
    dll.display();

    int originalSize = dll.getSize();

    cout << "Deleting from beginning and end" << endl;
    dll.deleteBegin();
    dll.deleteEnd();
    dll.display();

    int delPositions[] = { 3, 6, 9, 12 };

    for (int i = 0; i < 4; i++) {
        int adjustedPos = delPositions[i] - i;
        cout << "Deleting position " << adjustedPos << " (original
position " << delPositions[i] << ")" << endl;
        dll.deletePos(adjustedPos);
        dll.display();
    }

    dll.display();

    return 0;
}

```

Output:

Inserting A-J at end: A B C D E F G H I J

List (10 elements): A B C D E F G H I J

Inserting K-T at beginning: T S R Q P O N M L K

List (20 elements): K L M N O P Q R S T A B C D E F G H I J

Inserting U-Y before positions 4,8,12,16,20: U before 4 V before 8 W before 12 X before 16 Y before 20

Inserting Z at end

Final list after all insertions: List (26 elements): K L M U N O P V Q R S W T A B X C D E Y F G H I J Z

Deleting from beginning and end

List (24 elements): L M U N O P V Q R S W T A B X C D E Y F G H I J

Deleting position 3 (original position 3)

List (23 elements): L M N O P V Q R S W T A B X C D E Y F G H I J

Deleting position 5 (original position 6)

List (22 elements): L M N O V Q R S W T A B X C D E Y F G H I J

Deleting position 7 (original position 9)

List (21 elements): L M N O V Q S W T A B X C D E Y F G H I J

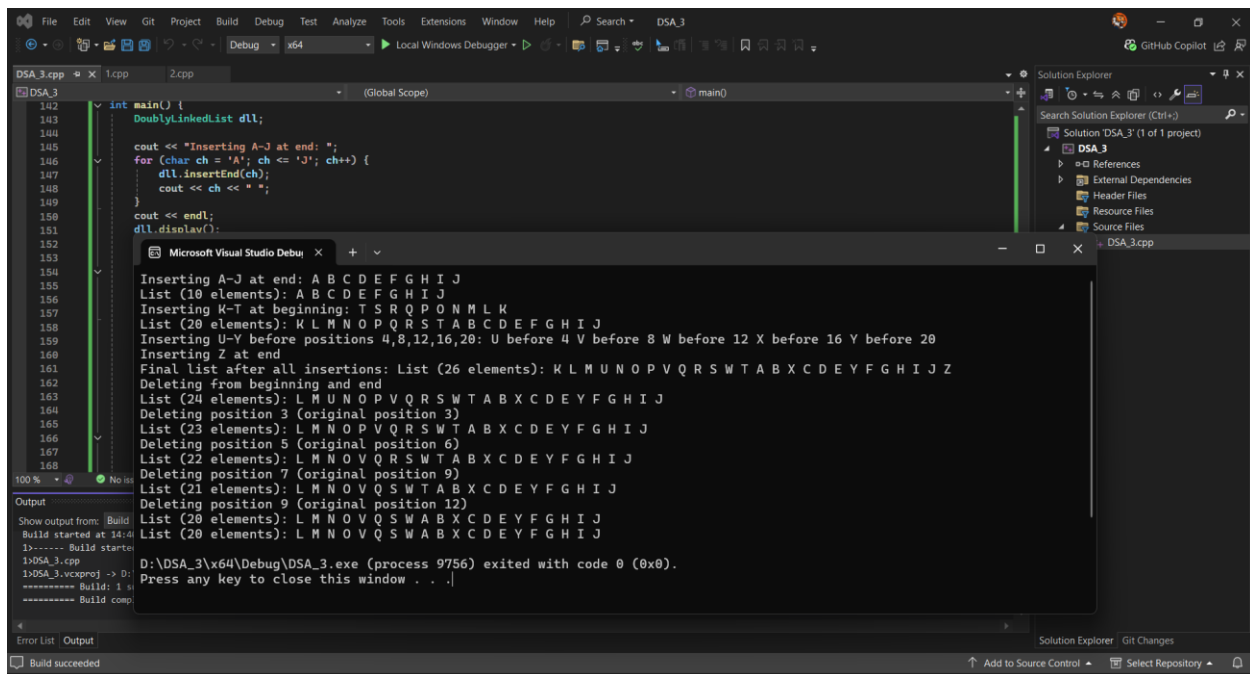
Deleting position 9 (original position 12)

List (20 elements): L M N O V Q S W A B X C D E Y F G H I J

List (20 elements): L M N O V Q S W A B X C D E Y F G H I J

D:\DSA_3\x64\Debug\DSA_3.exe (process 9756) exited with code 0 (0x0).

Press any key to close this window . . .



Sol12.

```
#include <iostream>

using namespace std;

struct Node {

    int data; Node* next;

    Node(int d) :data(d), next(NULL) {}

};

class SLL {

    Node* head = NULL;

public:

    Node* getHead() { return head; }

    void insertEnd(int d) {

        Node* n = new Node(d);
```

```

        if (!head) head = n;

        else { Node* t = head; while (t->next) t = t->next; t->next = n; }
    }

    void show() {

        for (Node* t = head; t; t = t->next) cout << t->data << " ";

        cout << "\n";

    }

    // Create list from array

    void build(int arr[], int n) { for (int i = 0; i < n; i++)
insertEnd(arr[i]); }

};

```

```

Node* Union(Node* a, Node* b) {

    Node dummy(0); Node* t = &dummy;

    while (a && b) {

        if (a->data == b->data) { t->next = new Node(a->data); a = a-
>next; b = b->next; }

        else if (a->data < b->data) { t->next = new Node(a->data); a = a-
>next; }

        else { t->next = new Node(b->data); b = b->next; }

        t = t->next;

    }

    while (a) { t->next = new Node(a->data); a = a->next; t = t->next; }

    while (b) { t->next = new Node(b->data); b = b->next; t = t->next; }

    return dummy.next;

}

```

```

Node* Intersection(Node* a, Node* b) {

    Node dummy(0); Node* t = &dummy;

```

```

while (a && b) {

    if (a->data == b->data) { t->next = new Node(a->data); t = t-
>next; a = a->next; b = b->next; }

    else if (a->data < b->data) a = a->next;

    else b = b->next;

}

return dummy.next;

}

void showList(Node* h) { while (h) { cout << h->data << " "; h = h->next;
} cout << "\n"; }

int main() {

    // Test Case 1 (Given)

    int A1[] = { 3,5,78,90,91,95,105,106,150,170,190,200,205,210 };

    int B1[] = { 2,3,5,98,106,107,190,201,204,222,250 };

    SLL s1, s2; s1.build(A1, 14); s2.build(B1, 11);

    cout << "S1: "; s1.show();

    cout << "S2: "; s2.show();

    Node* u1 = Union(s1.getHead(), s2.getHead());

    Node* i1 = Intersection(s1.getHead(), s2.getHead());

    cout << "Union: "; showList(u1);

    cout << "Intersection: "; showList(i1);

    // Test Case 2

```

```

int A2[] = { 1,4,7,10 };

int B2[] = { 2,4,6,10,12 };

SLL s3, s4; s3.build(A2, 4); s4.build(B2, 5);

cout << "\nS3: "; s3.show();

cout << "S4: "; s4.show();

cout << "Union: "; showList(Union(s3.getHead(), s4.getHead()));

cout << "Intersection: "; showList(Intersection(s3.getHead(),
s4.getHead()));

// Test Case 3

int A3[] = { 11,22,33,44 };

int B3[] = { 55,66,77 };

SLL s5, s6; s5.build(A3, 4); s6.build(B3, 3);

cout << "\nS5: "; s5.show();

cout << "S6: "; s6.show();

cout << "Union: "; showList(Union(s5.getHead(), s6.getHead()));

cout << "Intersection: "; showList(Intersection(s5.getHead(),
s6.getHead()));

}

```

Output:

S1: 3 5 78 90 91 95 105 106 150 170 190 200 205 210

S2: 2 3 5 98 106 107 190 201 204 222 250

Union: 2 3 5 78 90 91 95 98 105 106 107 150 170 190 200 201 204 205 210 222 250

Intersection: 3 5 106 190

S3: 1 4 7 10

S4: 2 4 6 10 12

Union: 1 2 4 6 7 10 12

Intersection: 4 10

S5: 11 22 33 44

S6: 55 66 77

Union: 11 22 33 44 55 66 77

Intersection:

D:\DSA_3\x64\Debug\DSA_3.exe (process 18372) exited with code 0 (0x0).

Press any key to close this window . . .

